

Data Science & AI

Complete Cheat Sheet

Pandas • NumPy • SQL • Excel • Matplotlib • Scikit-Learn • PyTorch • LangGraph

Functions, descriptions, and code examples for every major API

Pandas

I/O — Reading & Writing Data

Function / Syntax	Description	Code Example
<code>pd.read_csv()</code>	Read CSV file into DataFrame	<code>df = pd.read_csv('data.csv')</code>
<code>pd.read_excel()</code>	Read Excel file into DataFrame	<code>df = pd.read_excel('data.xlsx')</code>
<code>pd.read_json()</code>	Read JSON into DataFrame	<code>df = pd.read_json('data.json')</code>
<code>pd.read_sql()</code>	Read SQL query into DataFrame	<code>df = pd.read_sql(query, conn)</code>
<code>pd.read_parquet()</code>	Read Parquet file	<code>df = pd.read_parquet('data.parquet')</code>
<code>df.to_csv()</code>	Write DataFrame to CSV	<code>df.to_csv('out.csv', index=False)</code>
<code>df.to_excel()</code>	Write DataFrame to Excel	<code>df.to_excel('out.xlsx')</code>
<code>df.to_parquet()</code>	Write DataFrame to Parquet	<code>df.to_parquet('out.parquet')</code>
<code>df.to_dict()</code>	Convert DataFrame to dict	<code>d = df.to_dict(orient='records')</code>

Inspection & Exploration

Function / Syntax	Description	Code Example
<code>df.head(n)</code>	First n rows (default 5)	<code>df.head(10)</code>
<code>df.tail(n)</code>	Last n rows	<code>df.tail(5)</code>
<code>df.info()</code>	Column types, nulls, memory	<code>df.info()</code>
<code>df.describe()</code>	Summary stats (count, mean...)	<code>df.describe(include='all')</code>
<code>df.shape</code>	Tuple of (rows, columns)	<code>rows, cols = df.shape</code>
<code>df.dtypes</code>	Data type of each column	<code>df.dtypes</code>
<code>df.columns</code>	List of column names	<code>list(df.columns)</code>
<code>df.index</code>	Row index	<code>df.index</code>
<code>df.memory_usage()</code>	Memory used per column	<code>df.memory_usage(deep=True)</code>
<code>df.nunique()</code>	Count of unique values per col	<code>df.nunique()</code>

Function / Syntax	Description	Code Example
<code>df.sample(n)</code>	Random sample of n rows	<code>df.sample(5, random_state=42)</code>

Selection & Filtering

Function / Syntax	Description	Code Example
<code>df['col']</code>	Select single column (Series)	<code>df['age']</code>
<code>df[['a', 'b']]</code>	Select multiple columns	<code>df[['name', 'age']]</code>
<code>df.loc[row, col]</code>	Label-based indexing	<code>df.loc[0:5, 'age']</code>
<code>df.iloc[row, col]</code>	Integer-based indexing	<code>df.iloc[0:5, 1:3]</code>
<code>df.at[row, col]</code>	Single value by label	<code>df.at[0, 'name']</code>
<code>df.iat[row, col]</code>	Single value by position	<code>df.iat[0, 1]</code>
<code>df.query(expr)</code>	Filter using string expression	<code>df.query('age > 30 & city=="NY"')</code>
<code>df[df.col > val]</code>	Boolean mask filter	<code>df[df['age'] > 30]</code>
<code>df.filter()</code>	Filter by column/row labels	<code>df.filter(like='price')</code>
<code>df.where(cond)</code>	Keep values where True, else NaN	<code>df.where(df > 0)</code>

Cleaning & Transformation

Function / Syntax	Description	Code Example
<code>df.isnull()</code>	Boolean mask of null values	<code>df.isnull().sum()</code>
<code>df.notnull()</code>	Boolean mask of non-null values	<code>df.notnull()</code>
<code>df.dropna()</code>	Drop rows with any null	<code>df.dropna(subset=['age'])</code>
<code>df.fillna(val)</code>	Fill nulls with value/method	<code>df.fillna({'age': 0, 'name': 'N/A'})</code>
<code>df.interpolate()</code>	Interpolate missing numeric vals	<code>df['price'].interpolate()</code>
<code>df.duplicated()</code>	Boolean mask of duplicate rows	<code>df.duplicated(subset=['id'])</code>
<code>df.drop_duplicates()</code>	Remove duplicate rows	<code>df.drop_duplicates(keep='first')</code>
<code>df.rename()</code>	Rename columns/index	<code>df.rename(columns={'old': 'new'})</code>
<code>df.astype()</code>	Cast column to new type	<code>df['age'].astype(int)</code>
<code>df.replace()</code>	Replace values	<code>df.replace({'Y': 1, 'N': 0})</code>
<code>df.clip()</code>	Clip values to min/max	<code>df['score'].clip(0, 100)</code>
<code>df.drop()</code>	Drop rows or columns	<code>df.drop(columns=['col1', 'col2'])</code>
<code>df.reset_index()</code>	Reset row index	<code>df.reset_index(drop=True)</code>
<code>df.set_index()</code>	Set a column as index	<code>df.set_index('id')</code>

Aggregation, Merging & Reshaping

Function / Syntax	Description	Code Example
<code>df.sort_values()</code>	Sort by column values	<code>df.sort_values('age', ascending=False)</code>
<code>df.sort_index()</code>	Sort by index	<code>df.sort_index()</code>
<code>df.groupby()</code>	Group rows & aggregate	<code>df.groupby('city')['salary'].mean()</code>

Function / Syntax	Description	Code Example
<code>df.agg()</code>	Apply multiple aggregations	<code>df.groupby('dept').agg({'sal':['mean','max']})</code>
<code>df.pivot_table()</code>	Create a pivot table	<code>pd.pivot_table(df, values='sal', index='dept')</code>
<code>df.melt()</code>	Wide to long format	<code>df.melt(id_vars=['id'], value_vars=['q1','q2'])</code>
<code>df.pivot()</code>	Long to wide format	<code>df.pivot(index='date', columns='item', values='val')</code>
<code>df.apply()</code>	Apply function along axis	<code>df['bmi'].apply(lambda x: 'high' if x>25 else 'ok')</code>
<code>df.applymap()</code>	Apply function element-wise	<code>df.applymap(str.upper)</code>
<code>df.map()</code>	Map values via dict/function	<code>df['grade'].map({'A':4, 'B':3, 'C':2})</code>
<code>df.transform()</code>	Transform keeping shape	<code>df.groupby('dept')['sal'].transform('mean')</code>
<code>df.merge()</code>	SQL-style join two DataFrames	<code>pd.merge(df1, df2, on='id', how='left')</code>
<code>df.join()</code>	Join on index	<code>df1.join(df2, how='inner')</code>
<code>pd.concat()</code>	Stack DataFrames row/col-wise	<code>pd.concat([df1, df2], ignore_index=True)</code>
<code>df.assign()</code>	Add/overwrite columns fluently	<code>df.assign(full_name=df.first+' '+df.last)</code>
<code>df.eval()</code>	Evaluate expression string	<code>df.eval('profit = revenue - cost')</code>
<code>df.explode()</code>	Explode list column to rows	<code>df.explode('tags')</code>
<code>df.stack()</code>	Pivot columns to rows	<code>df.stack()</code>
<code>df.unstack()</code>	Pivot rows to columns	<code>df.unstack()</code>
<code>df.value_counts()</code>	Frequency count	<code>df['city'].value_counts(normalize=True)</code>
<code>df.cut()</code>	Bin continuous data	<code>pd.cut(df['age'], bins=[0,18,65,100])</code>
<code>df.get_dummies()</code>	One-hot encode categoricals	<code>pd.get_dummies(df['color'])</code>
<code>df.crosstab()</code>	Cross-tabulation of two cols	<code>pd.crosstab(df['dept'], df['gender'])</code>

String Methods (df.str.*)

Function / Syntax	Description	Code Example
<code>df.str.lower()</code>	Lowercase all strings	<code>df['name'].str.lower()</code>
<code>df.str.upper()</code>	Uppercase all strings	<code>df['name'].str.upper()</code>
<code>df.str.strip()</code>	Strip whitespace	<code>df['name'].str.strip()</code>
<code>df.str.contains()</code>	Check if string contains pattern	<code>df['email'].str.contains('@gmail')</code>
<code>df.str.replace()</code>	Replace substring	<code>df['text'].str.replace('old', 'new')</code>
<code>df.str.split()</code>	Split string to list	<code>df['full_name'].str.split(' ', expand=True)</code>
<code>df.str.len()</code>	Length of each string	<code>df['name'].str.len()</code>
<code>df.str.extract()</code>	Extract regex groups	<code>df['phone'].str.extract(r'(\d{3})')</code>
<code>df.str.startswith()</code>	Check prefix	<code>df['col'].str.startswith('pre')</code>

DateTime Methods (df.dt.*)

Function / Syntax	Description	Code Example
<code>pd.to_datetime()</code>	Parse string to datetime	<code>pd.to_datetime(df['date'])</code>
<code>dt.year / .month / .day</code>	Extract date components	<code>df['date'].dt.year</code>
<code>dt.dayofweek</code>	Day of week (0=Mon)	<code>df['date'].dt.dayofweek</code>
<code>dt.hour / .minute</code>	Extract time components	<code>df['ts'].dt.hour</code>
<code>df.resample()</code>	Resample time series	<code>df.resample('M').mean()</code>
<code>df.shift()</code>	Shift values by periods	<code>df['price'].shift(1)</code>
<code>df.diff()</code>	Difference between rows	<code>df['price'].diff()</code>
<code>df.rolling(n)</code>	Rolling window calculation	<code>df['price'].rolling(7).mean()</code>
<code>df.expanding()</code>	Expanding window	<code>df['price'].expanding().mean()</code>
<code>df.ewm()</code>	Exponentially weighted mean	<code>df['price'].ewm(span=7).mean()</code>

NumPy

Array Creation

Function / Syntax	Description	Code Example
<code>np.array()</code>	Create array from list/tuple	<code>np.array([[1,2],[3,4]])</code>
<code>np.zeros(shape)</code>	Array of zeros	<code>np.zeros((3, 4))</code>
<code>np.ones(shape)</code>	Array of ones	<code>np.ones((3, 4), dtype=float)</code>
<code>np.full(shape, val)</code>	Fill array with value	<code>np.full((2,3), 7)</code>
<code>np.eye(n)</code>	Identity matrix	<code>np.eye(4)</code>
<code>np.arange()</code>	Evenly spaced int range	<code>np.arange(0, 10, 2)</code>
<code>np.linspace()</code>	Evenly spaced floats	<code>np.linspace(0, 1, 100)</code>
<code>np.logspace()</code>	Log-spaced values	<code>np.logspace(0, 2, 5)</code>
<code>np.random.rand()</code>	Uniform random [0, 1)	<code>np.random.rand(3, 4)</code>
<code>np.random.randn()</code>	Standard normal random	<code>np.random.randn(100)</code>
<code>np.random.randint()</code>	Random integers	<code>np.random.randint(0, 10, size=(3,3))</code>
<code>np.random.choice()</code>	Random sample from array	<code>np.random.choice(arr, size=5)</code>
<code>np.random.seed()</code>	Set random seed	<code>np.random.seed(42)</code>
<code>np.empty(shape)</code>	Uninitialized array (fast)	<code>np.empty((3,3))</code>

Shape & Manipulation

Function / Syntax	Description	Code Example
<code>arr.shape</code>	Tuple of dimensions	<code>arr.shape # (100, 4)</code>
<code>arr.ndim</code>	Number of dimensions	<code>arr.ndim # 2</code>
<code>arr.dtype</code>	Data type of elements	<code>arr.dtype # float64</code>
<code>arr.size</code>	Total number of elements	<code>arr.size # 400</code>
<code>arr.reshape()</code>	Reshape without copying	<code>arr.reshape(20, 20)</code>

Function / Syntax	Description	Code Example
<code>arr.ravel()</code>	Flatten to 1D (view)	<code>arr.ravel()</code>
<code>arr.flatten()</code>	Flatten to 1D (copy)	<code>arr.flatten()</code>
<code>arr.transpose()</code>	Transpose axes	<code>arr.T</code> # or <code>arr.transpose()</code>
<code>np.expand_dims()</code>	Add new axis	<code>np.expand_dims(arr, axis=0)</code>
<code>np.squeeze()</code>	Remove size-1 axes	<code>np.squeeze(arr.reshape(1,4,1))</code>
<code>np.concatenate()</code>	Join arrays along axis	<code>np.concatenate([a, b], axis=0)</code>
<code>np.stack()</code>	Stack along new axis	<code>np.stack([a, b], axis=1)</code>
<code>np.hstack()</code>	Stack horizontally	<code>np.hstack([a, b])</code>
<code>np.vstack()</code>	Stack vertically	<code>np.vstack([a, b])</code>
<code>np.split()</code>	Split array	<code>np.split(arr, 3)</code>

Math & Statistics

Function / Syntax	Description	Code Example
<code>np.sum()</code>	Sum all or along axis	<code>np.sum(arr, axis=0)</code>
<code>np.mean()</code>	Mean	<code>np.mean(arr, axis=1)</code>
<code>np.median()</code>	Median	<code>np.median(arr)</code>
<code>np.std()</code>	Standard deviation	<code>np.std(arr, ddof=1)</code>
<code>np.var()</code>	Variance	<code>np.var(arr)</code>
<code>np.min()</code> / <code>np.max()</code>	Min / Max value	<code>arr.min(axis=0)</code>
<code>np.argmin()</code> / <code>np.argmax()</code>	Index of min / max	<code>np.argmin(arr)</code>
<code>np.cumsum()</code>	Cumulative sum	<code>np.cumsum(arr)</code>
<code>np.cumprod()</code>	Cumulative product	<code>np.cumprod(arr)</code>
<code>np.abs()</code>	Absolute value	<code>np.abs(arr)</code>
<code>np.sqrt()</code>	Square root	<code>np.sqrt(arr)</code>
<code>np.exp()</code>	Exponential (e^x)	<code>np.exp(arr)</code>
<code>np.log()</code>	Natural log	<code>np.log(arr)</code>
<code>np.log2()</code> / <code>np.log10()</code>	Log base 2 / 10	<code>np.log10(arr)</code>
<code>np.power()</code>	Raise to power	<code>np.power(arr, 2)</code>
<code>np.dot()</code>	Dot product / matrix mul	<code>np.dot(A, B)</code>
<code>np.matmul()</code> / <code>@</code>	Matrix multiplication	<code>A @ B</code>
<code>np.clip()</code>	Clip values to range	<code>np.clip(arr, 0, 1)</code>
<code>np.round()</code>	Round to n decimals	<code>np.round(arr, 2)</code>
<code>np.floor()</code> / <code>np.ceil()</code>	Floor / Ceiling	<code>np.floor(arr)</code>
<code>np.where()</code>	Conditional select	<code>np.where(arr > 0, arr, 0)</code>
<code>np.unique()</code>	Unique values + counts	<code>np.unique(arr, return_counts=True)</code>
<code>np.sort()</code>	Sort array	<code>np.sort(arr, axis=0)</code>
<code>np.argsort()</code>	Indices that sort array	<code>np.argsort(arr)</code>
<code>np.percentile()</code>	Percentile value	<code>np.percentile(arr, 75)</code>
<code>np.corrcoef()</code>	Correlation matrix	<code>np.corrcoef(x, y)</code>

Function / Syntax	Description	Code Example
<code>np.cov()</code>	Covariance matrix	<code>np.cov(X.T)</code>

Linear Algebra

Function / Syntax	Description	Code Example
<code>np.linalg.inv()</code>	Matrix inverse	<code>np.linalg.inv(A)</code>
<code>np.linalg.det()</code>	Determinant	<code>np.linalg.det(A)</code>
<code>np.linalg.eig()</code>	Eigenvalues & eigenvectors	<code>vals, vecs = np.linalg.eig(A)</code>
<code>np.linalg.svd()</code>	Singular Value Decomposition	<code>U, S, V = np.linalg.svd(A)</code>
<code>np.linalg.norm()</code>	Vector / matrix norm	<code>np.linalg.norm(arr)</code>
<code>np.linalg.solve()</code>	Solve linear system Ax=b	<code>x = np.linalg.solve(A, b)</code>
<code>np.linalg.rank()</code>	Matrix rank	<code>np.linalg.matrix_rank(A)</code>
<code>np.linalg.pinv()</code>	Pseudo-inverse	<code>np.linalg.pinv(A)</code>



SQL

Basic Querying

Function / Syntax	Description	Code Example
<code>SELECT col1, col2</code>	Choose columns to return	<code>SELECT name, age FROM users</code>
<code>SELECT *</code>	Select all columns	<code>SELECT * FROM orders</code>
<code>SELECT DISTINCT</code>	Remove duplicates	<code>SELECT DISTINCT city FROM customers</code>
<code>FROM table</code>	Source table	<code>SELECT * FROM employees</code>
<code>WHERE condition</code>	Filter rows	<code>WHERE salary > 50000 AND dept = 'Eng'</code>
<code>AND / OR / NOT</code>	Logical operators	<code>WHERE age > 18 AND (city='NY' OR city='LA')</code>
<code>IN (list)</code>	Match any value in list	<code>WHERE country IN ('US','UK','CA')</code>
<code>NOT IN (list)</code>	Exclude values in list	<code>WHERE status NOT IN ('closed','cancelled')</code>
<code>BETWEEN a AND b</code>	Range filter (inclusive)	<code>WHERE date BETWEEN '2024-01-01' AND '2024-12-31'</code>
<code>LIKE pattern</code>	Pattern matching	<code>WHERE email LIKE '%@gmail.com'</code>
<code>IS NULL / IS NOT NULL</code>	Null checks	<code>WHERE phone IS NOT NULL</code>
<code>ORDER BY col</code>	Sort results	<code>ORDER BY salary DESC, name ASC</code>
<code>LIMIT n</code>	Limit result rows	<code>SELECT * FROM logs ORDER BY ts DESC LIMIT 100</code>
<code>OFFSET n</code>	Skip n rows (pagination)	<code>LIMIT 10 OFFSET 20</code>
<code>AS alias</code>	Rename column/table	<code>SELECT salary * 1.1 AS adjusted_salary</code>

Aggregation & Grouping

Function / Syntax	Description	Code Example
<code>COUNT(*)</code>	Count all rows	<code>SELECT COUNT(*) FROM orders</code>
<code>COUNT(col)</code>	Count non-null values	<code>SELECT COUNT(DISTINCT user_id) FROM orders</code>
<code>SUM(col)</code>	Sum of column	<code>SELECT SUM(amount) FROM payments</code>
<code>AVG(col)</code>	Average of column	<code>SELECT AVG(score) FROM results</code>
<code>MIN(col) / MAX(col)</code>	Min / Max value	<code>SELECT MIN(price), MAX(price) FROM products</code>
<code>GROUP BY col</code>	Group rows for aggregation	<code>SELECT dept, AVG(salary) FROM emp GROUP BY dept</code>
<code>HAVING condition</code>	Filter groups (after GROUP BY)	<code>HAVING COUNT(*) > 5 AND AVG(sal) > 60000</code>
<code>ROLLUP</code>	Subtotals + grand total	<code>GROUP BY dept, role WITH ROLLUP</code>
<code>GROUPING SETS</code>	Custom grouping combinations	<code>GROUP BY GROUPING SETS((dept), (role), ())</code>

Joins & Set Operations

Function / Syntax	Description	Code Example
<code>INNER JOIN</code>	Rows matching in both tables	<code>FROM a INNER JOIN b ON a.id = b.id</code>
<code>LEFT JOIN</code>	All left + matched right	<code>FROM a LEFT JOIN b ON a.id = b.id</code>
<code>RIGHT JOIN</code>	All right + matched left	<code>FROM a RIGHT JOIN b ON a.id = b.id</code>
<code>FULL OUTER JOIN</code>	All rows from both tables	<code>FROM a FULL OUTER JOIN b ON a.id = b.id</code>
<code>CROSS JOIN</code>	Cartesian product	<code>FROM colors CROSS JOIN sizes</code>
<code>SELF JOIN</code>	Join table with itself	<code>FROM emp e1 JOIN emp e2 ON e1.mgr_id = e2.id</code>
<code>UNION</code>	Combine + deduplicate results	<code>SELECT city FROM a UNION SELECT city FROM b</code>
<code>UNION ALL</code>	Combine keeping duplicates	<code>SELECT id FROM a UNION ALL SELECT id FROM b</code>
<code>INTERSECT</code>	Rows in both result sets	<code>SELECT id FROM a INTERSECT SELECT id FROM b</code>
<code>EXCEPT / MINUS</code>	Rows in first but not second	<code>SELECT id FROM a EXCEPT SELECT id FROM b</code>

Window Functions

Function / Syntax	Description	Code Example
<code>ROW_NUMBER()</code>	Sequential row number	<code>ROW_NUMBER() OVER (PARTITION BY dept ORDER BY sal DESC)</code>
<code>RANK()</code>	Rank with gaps for ties	<code>RANK() OVER (ORDER BY score DESC)</code>
<code>DENSE_RANK()</code>	Rank without gaps	<code>DENSE_RANK() OVER (PARTITION BY dept ORDER BY sal)</code>
<code>NTILE(n)</code>	Divide into n equal buckets	<code>NTILE(4) OVER (ORDER BY salary)</code>
<code>LAG(col, n)</code>	Value from n rows before	<code>LAG(price, 1) OVER (ORDER BY date)</code>
<code>LEAD(col, n)</code>	Value from n rows ahead	<code>LEAD(price, 1) OVER (ORDER BY date)</code>
<code>FIRST_VALUE()</code>	First value in window	<code>FIRST_VALUE(price) OVER (PARTITION BY id ORDER BY date)</code>

Function / Syntax	Description	Code Example
<code>LAST_VALUE()</code>	Last value in window	<code>LAST_VALUE(price) OVER (PARTITION BY id ORDER BY date)</code>
<code>SUM() OVER ()</code>	Running total	<code>SUM(amount) OVER (ORDER BY date)</code>
<code>AVG() OVER ()</code>	Moving average	<code>AVG(price) OVER (ORDER BY date ROWS 6 PRECEDING)</code>
<code>PARTITION BY</code>	Define window partitions	<code>OVER (PARTITION BY dept ORDER BY salary)</code>

Advanced — CTEs, Subqueries, Functions

Function / Syntax	Description	Code Example
<code>WITH cte AS (...)</code>	Common Table Expression	<code>WITH top AS (SELECT * FROM emp ORDER BY sal DESC LIMIT 10)</code>
<code>CASE WHEN</code>	Conditional logic	<code>CASE WHEN sal>100k THEN 'high' ELSE 'low' END</code>
<code>COALESCE(a, b)</code>	First non-null value	<code>COALESCE(phone, email, 'N/A')</code>
<code>NULLIF(a, b)</code>	Return null if a equals b	<code>NULLIF(score, 0)</code>
<code>CAST(col AS type)</code>	Type conversion	<code>CAST(price AS DECIMAL(10,2))</code>
<code>SUBSTRING()</code>	Extract part of string	<code>SUBSTRING(name, 1, 3)</code>
<code>CONCAT()</code>	Join strings	<code>CONCAT(first_name, ' ', last_name)</code>
<code>DATE_TRUNC()</code>	Truncate date to unit	<code>DATE_TRUNC('month', created_at)</code>
<code>DATE_DIFF()</code>	Difference between dates	<code>DATEDIFF('2024-12-31', '2024-01-01')</code>
<code>EXTRACT()</code>	Extract date part	<code>EXTRACT(YEAR FROM order_date)</code>
Subquery	Query inside query	<code>WHERE id IN (SELECT id FROM vip_users)</code>
<code>EXISTS</code>	Check if subquery has rows	<code>WHERE EXISTS (SELECT 1 FROM orders WHERE user_id=u.id)</code>
<code>CREATE INDEX</code>	Speed up queries	<code>CREATE INDEX idx_email ON users(email)</code>
<code>EXPLAIN</code>	Show query execution plan	<code>EXPLAIN SELECT * FROM large_table WHERE id > 100</code>



Lookup & Reference

Function / Syntax	Description	Code Example
<code>VLOOKUP(val, range, col, 0)</code>	Vertical exact match lookup	<code>=VLOOKUP(A2, Sheet2!A:C, 2, 0)</code>
<code>XLOOKUP(val, arr, ret)</code>	Modern flexible lookup	<code>=XLOOKUP(A2, B:B, C:C, "Not Found")</code>
<code>INDEX(range, row, col)</code>	Return value at position	<code>=INDEX(A1:D10, 3, 2)</code>
<code>MATCH(val, range, 0)</code>	Position of value in range	<code>=MATCH("Bob", A:A, 0)</code>
<code>INDEX+MATCH</code>	Two-way flexible lookup	<code>=INDEX(C:C, MATCH(A2, B:B, 0))</code>
<code>HLOOKUP(val, range, row)</code>	Horizontal lookup	<code>=HLOOKUP("Q1", A1:D5, 3, 0)</code>
<code>CHOOSE(idx, v1, v2)</code>	Return value by index	<code>=CHOOSE(2, "A", "B", "C")</code>
<code>OFFSET(ref, rows, cols)</code>	Dynamic cell reference	<code>=OFFSET(A1, 3, 2)</code>

Function / Syntax	Description	Code Example
<code>INDIRECT (ref_text)</code>	Reference from text string	<code>=INDIRECT ("Sheet2!A"&B1)</code>

Math & Statistical

Function / Syntax	Description	Code Example
<code>SUM (range)</code>	Sum values	<code>=SUM (A1:A100)</code>
<code>SUMIF (range, crit, sum)</code>	Sum with condition	<code>=SUMIF (B:B, "NY", C:C)</code>
<code>SUMIFS ()</code>	Sum with multiple conditions	<code>=SUMIFS (D:D, B:B, "NY", C:C, ">1000")</code>
<code>AVERAGE (range)</code>	Mean of values	<code>=AVERAGE (B2:B50)</code>
<code>AVERAGEIF ()</code>	Average with condition	<code>=AVERAGEIF (A:A, ">18", B:B)</code>
<code>COUNT (range)</code>	Count numeric cells	<code>=COUNT (A1:A100)</code>
<code>COUNTA (range)</code>	Count non-empty cells	<code>=COUNTA (A:A) -1</code>
<code>COUNTIF (range, crit)</code>	Count with condition	<code>=COUNTIF (B:B, "NY")</code>
<code>COUNTIFS ()</code>	Count with multiple conditions	<code>=COUNTIFS (A:A, ">18", B:B, "NY")</code>
<code>MIN () / MAX ()</code>	Smallest / largest value	<code>=MAX (A1:A100)</code>
<code>LARGE (range, k)</code>	k-th largest value	<code>=LARGE (A1:A100, 3)</code>
<code>SMALL (range, k)</code>	k-th smallest value	<code>=SMALL (A1:A100, 1)</code>
<code>RANK (val, range)</code>	Rank of value in range	<code>=RANK (A2, A:A, 0)</code>
<code>ROUND (val, digits)</code>	Round to decimal places	<code>=ROUND (3.14159, 2)</code>
<code>INT (val)</code>	Round down to integer	<code>=INT (4.9) // returns 4</code>
<code>MOD (num, div)</code>	Remainder after division	<code>=MOD (10, 3) // returns 1</code>
<code>ABS (val)</code>	Absolute value	<code>=ABS (-42)</code>
<code>POWER (base, exp)</code>	Raise to power	<code>=POWER (2, 10) // 1024</code>
<code>SQRT (val)</code>	Square root	<code>=SQRT (144)</code>
<code>PRODUCT (range)</code>	Multiply all values	<code>=PRODUCT (A1:A5)</code>

Text Manipulation

Function / Syntax	Description	Code Example
<code>CONCATENATE ()</code>	Join text (legacy)	<code>=CONCATENATE (A2, " ", B2)</code>
<code>CONCAT ()</code>	Join text (modern)	<code>=CONCAT (A2:C2)</code>
<code>TEXTJOIN (del, T, range)</code>	Join with delimiter	<code>=TEXTJOIN (" ", TRUE, A1:A5)</code>
<code>LEFT (text, n)</code>	Extract left n chars	<code>=LEFT (A2, 3)</code>
<code>RIGHT (text, n)</code>	Extract right n chars	<code>=RIGHT (A2, 4)</code>
<code>MID (text, start, n)</code>	Extract from middle	<code>=MID (A2, 5, 3)</code>
<code>LEN (text)</code>	String length	<code>=LEN (A2)</code>
<code>TRIM (text)</code>	Remove extra spaces	<code>=TRIM (A2)</code>
<code>UPPER () / LOWER ()</code>	Change case	<code>=UPPER (A2)</code>
<code>PROPER (text)</code>	Title Case	<code>=PROPER ("john doe")</code>
<code>SUBSTITUTE (t, old, new)</code>	Replace substring	<code>=SUBSTITUTE (A2, "-", "/")</code>

Function / Syntax	Description	Code Example
<code>FIND (sub , text)</code>	Position of substring	<code>=FIND ("@" , A2)</code>
<code>TEXT (val , format)</code>	Format number as text	<code>=TEXT (A2 , "\$# , ##0.00")</code>
<code>VALUE (text)</code>	Convert text to number	<code>=VALUE ("42")</code>
<code>REPT (text , n)</code>	Repeat text n times	<code>=REPT ("*", 5)</code>

Logic & Conditionals

Function / Syntax	Description	Code Example
<code>IF (test , true , false)</code>	Conditional value	<code>=IF (A2>100 , "High" , "Low")</code>
<code>IFS (t1 , v1 , t2 , v2)</code>	Multiple conditions	<code>=IFS (A2>90 , "A" , A2>80 , "B" , TRUE , "C")</code>
<code>AND (cond1 , cond2)</code>	All conditions true	<code>=IF (AND (A2>0 , B2>0) , "Both" , "No")</code>
<code>OR (cond1 , cond2)</code>	Any condition true	<code>=IF (OR (A2="NY" , A2="LA") , "Coast" , "Mid")</code>
<code>NOT (condition)</code>	Negate condition	<code>=IF (NOT (ISBLANK (A2)) , A2 , "Empty")</code>
<code>IFERROR (val , err)</code>	Handle formula errors	<code>=IFERROR (A2/B2 , 0)</code>
<code>IFNA (val , na_val)</code>	Handle #N/A errors	<code>=IFNA (VLOOKUP (A2 , B:C , 2 , 0) , "Not found")</code>
<code>SWITCH (expr , v1 , r1)</code>	Switch statement	<code>=SWITCH (A2 , 1 , "Jan" , 2 , "Feb" , "Other")</code>
<code>ISBLANK (cell)</code>	Check if cell is empty	<code>=IF (ISBLANK (A2) , "Empty" , "Filled")</code>
<code>ISNUMBER (cell)</code>	Check if value is number	<code>=ISNUMBER (A2)</code>

Date & Time

Function / Syntax	Description	Code Example
<code>TODAY ()</code>	Current date	<code>=TODAY ()</code>
<code>NOW ()</code>	Current date & time	<code>=NOW ()</code>
<code>DATE (y , m , d)</code>	Build a date	<code>=DATE (2024 , 12 , 31)</code>
<code>YEAR () / MONTH () / DAY ()</code>	Extract date parts	<code>=YEAR (A2)</code>
<code>WEEKDAY (date)</code>	Day of week (1=Sun)	<code>=WEEKDAY (TODAY ())</code>
<code>NETWORKDAYS (s , e)</code>	Working days between dates	<code>=NETWORKDAYS (A2 , B2)</code>
<code>WORKDAY (date , n)</code>	Date n working days out	<code>=WORKDAY (TODAY () , 30)</code>
<code>EDATE (date , months)</code>	Add months to date	<code>=EDATE (A2 , 6)</code>
<code>EOMONTH (date , n)</code>	Last day of nth month	<code>=EOMONTH (TODAY () , 0)</code>
<code>DATEDIF (s , e , unit)</code>	Difference between dates	<code>=DATEDIF (A2 , TODAY () , "Y")</code>

Dynamic Array Functions (Excel 365)

Function / Syntax	Description	Code Example
<code>FILTER (arr , cond)</code>	Dynamic filtered range	<code>=FILTER (A:B , C:C="NY")</code>
<code>SORT (range , col , ord)</code>	Dynamic sorted range	<code>=SORT (A1:C10 , 2 , -1)</code>
<code>SORTBY (range , by)</code>	Sort by another range	<code>=SORTBY (A:B , C:C , -1)</code>
<code>UNIQUE (range)</code>	Unique values list	<code>=UNIQUE (A2:A100)</code>

Function / Syntax	Description	Code Example
<code>SEQUENCE(r,c,start)</code>	Generate number sequence	<code>=SEQUENCE(12,1,1,1)</code>
<code>RANDARRAY(r,c)</code>	Random number array	<code>=RANDARRAY(5,3,1,100,TRUE)</code>
<code>TOCOL(range)</code>	Convert range to column	<code>=TOCOL(A1:D5)</code>
<code>TOROW(range)</code>	Convert range to row	<code>=TOROW(A1:A5)</code>
<code>VSTACK(r1,r2)</code>	Stack ranges vertically	<code>=VSTACK(A1:C5,Sheet2!A1:C3)</code>
<code>HSTACK(r1,r2)</code>	Stack ranges horizontally	<code>=HSTACK(A1:B5,D1:E5)</code>
<code>BYROW(range,fn)</code>	Apply LAMBDA to each row	<code>=BYROW(A1:C3,LAMBDA(r,SUM(r)))</code>
<code>LAMBDA(x,formula)</code>	Create custom function	<code>=LAMBDA(x,x*x)(5) // 25</code>

Matplotlib

Chart Types

Function / Syntax	Description	Code Example
<code>plt.plot(x, y)</code>	Line chart	<code>plt.plot(x, y, color='blue', linewidth=2)</code>
<code>plt.scatter(x, y)</code>	Scatter plot	<code>plt.scatter(x, y, c=colors, s=sizes, alpha=0.7)</code>
<code>plt.bar(x, height)</code>	Vertical bar chart	<code>plt.bar(cats, vals, color='steelblue', edgecolor='black')</code>
<code>plt.barh(y, width)</code>	Horizontal bar chart	<code>plt.barh(labels, values)</code>
<code>plt.hist(x, bins)</code>	Histogram	<code>plt.hist(data, bins=30, density=True, alpha=0.7)</code>
<code>plt.pie(sizes)</code>	Pie chart	<code>plt.pie(sizes, labels=lbls, autopct='%1.1f%%')</code>
<code>plt.boxplot(data)</code>	Box & whisker plot	<code>plt.boxplot([grp1, grp2], labels=['A','B'])</code>
<code>plt.violinplot(data)</code>	Violin plot	<code>plt.violinplot(data, showmeans=True)</code>
<code>plt.stem(x, y)</code>	Stem plot	<code>plt.stem(x, y)</code>
<code>plt.step(x, y)</code>	Step / staircase plot	<code>plt.step(x, y, where='post')</code>
<code>plt.fill_between(x,y1,y2)</code>	Shade area between lines	<code>plt.fill_between(x, y1, y2, alpha=0.3)</code>
<code>plt.errorbar(x,y,yerr)</code>	Error bar plot	<code>plt.errorbar(x, y, yerr=err, fmt='o')</code>
<code>plt.stackplot(x, y1,y2)</code>	Stacked area chart	<code>plt.stackplot(x, y1, y2, labels=['A','B'])</code>

2D Array & Field Plots

Function / Syntax	Description	Code Example
<code>plt.imshow(data)</code>	Display 2D array as image	<code>plt.imshow(matrix, cmap='viridis', aspect='auto')</code>
<code>plt.colorbar()</code>	Add colorbar legend	<code>plt.colorbar(label='Intensity')</code>
<code>plt.contour()</code>	Contour lines	<code>plt.contour(X, Y, Z, levels=10)</code>
<code>plt.contourf()</code>	Filled contour plot	<code>plt.contourf(X, Y, Z, cmap='RdBu')</code>

Function / Syntax	Description	Code Example
<code>plt.quiver(X,Y,U,V)</code>	Vector field / arrows	<code>plt.quiver(X, Y, U, V)</code>
<code>plt.hexbin(x,y)</code>	Hexagonal bin plot	<code>plt.hexbin(x, y, gridsize=30, cmap='Blues')</code>

Figures, Subplots & Layout

Function / Syntax	Description	Code Example
<code>plt.figure(figsize)</code>	Create figure with size	<code>fig = plt.figure(figsize=(10, 6))</code>
<code>plt.subplots(r,c)</code>	Grid of subplots	<code>fig, axes = plt.subplots(2, 3, figsize=(12,8))</code>
<code>plt.subplot(r,c,i)</code>	Select subplot by index	<code>plt.subplot(1, 2, 1)</code>
<code>plt.subplot2grid()</code>	Complex grid layout	<code>plt.subplot2grid((3,3),(0,0),colspan=2)</code>
<code>ax.twinx()</code>	Second y-axis on same plot	<code>ax2 = ax.twinx()</code>
<code>plt.tight_layout()</code>	Auto-fix layout overlap	<code>plt.tight_layout(pad=2.0)</code>
<code>plt.subplots_adjust()</code>	Manual subplot spacing	<code>plt.subplots_adjust(hspace=0.4)</code>
<code>plt.GridSpec()</code>	Fine-grained grid control	<code>gs = GridSpec(2, 2, width_ratios=[1,2])</code>

Labels, Annotations & Styling

Function / Syntax	Description	Code Example
<code>plt.xlabel(text)</code>	X-axis label	<code>plt.xlabel('Time (s)', fontsize=12)</code>
<code>plt.ylabel(text)</code>	Y-axis label	<code>plt.ylabel('Price (\$)')</code>
<code>plt.title(text)</code>	Plot title	<code>plt.title('Sales Over Time', fontsize=14, fontweight='bold')</code>
<code>plt.legend()</code>	Show legend	<code>plt.legend(loc='upper right', framealpha=0.9)</code>
<code>plt.xlim()</code> / <code>ylim()</code>	Set axis limits	<code>plt.xlim(0, 100); plt.ylim(-1, 1)</code>
<code>plt.xticks()</code> / <code>yticks()</code>	Custom tick marks	<code>plt.xticks(range(0,100,10), rotation=45)</code>
<code>plt.grid()</code>	Add grid lines	<code>plt.grid(True, linestyle='--', alpha=0.5)</code>
<code>plt.axhline(y)</code>	Horizontal line	<code>plt.axhline(y=0, color='red', linestyle='--')</code>
<code>plt.axvline(x)</code>	Vertical line	<code>plt.axvline(x=50, color='green')</code>
<code>plt.annotate(text)</code>	Annotate with arrow	<code>plt.annotate('Peak', xy=(5,10), xytext=(7,12), arrowprops={})</code>
<code>plt.text(x, y, s)</code>	Add text at position	<code>plt.text(0.5, 0.9, 'Label', transform=ax.transAxes)</code>
<code>plt.savefig(fname)</code>	Save to file	<code>plt.savefig('chart.png', dpi=150, bbox_inches='tight')</code>
<code>plt.show()</code>	Render and display	<code>plt.show()</code>
<code>plt.style.use()</code>	Apply a style theme	<code>plt.style.use('seaborn-v0_8-darkgrid')</code>
<code>plt.rcParams[]</code>	Set global defaults	<code>plt.rcParams['font.size'] = 12</code>



Preprocessing & Encoding

Function / Syntax	Description	Code Example
<code>train_test_split()</code>	Split into train/test sets	<pre>X_tr,X_te,y_tr,y_te = train_test_split(X,y,test_size=0.2)</pre>
<code>StandardScaler()</code>	Standardize (zero mean, unit var)	<pre>scaler = StandardScaler(); X_sc = scaler.fit_transform(X)</pre>
<code>MinMaxScaler()</code>	Scale to [0, 1]	<pre>MinMaxScaler().fit_transform(X)</pre>
<code>RobustScaler()</code>	Scale robust to outliers	<pre>RobustScaler().fit_transform(X)</pre>
<code>Normalizer()</code>	Scale each sample to unit norm	<pre>Normalizer().fit_transform(X)</pre>
<code>LabelEncoder()</code>	Encode target labels as int	<pre>le = LabelEncoder(); y = le.fit_transform(y)</pre>
<code>OrdinalEncoder()</code>	Encode ordinal categories	<pre>OrdinalEncoder().fit_transform(X)</pre>
<code>OneHotEncoder()</code>	One-hot encode categories	<pre>OneHotEncoder(sparse=False).fit_transform(X)</pre>
<code>LabelBinarizer()</code>	Binarize multi-class labels	<pre>LabelBinarizer().fit_transform(y)</pre>
<code>SimpleImputer()</code>	Fill missing values	<pre>SimpleImputer(strategy='mean').fit_transform(X)</pre>
<code>PolynomialFeatures()</code>	Create polynomial features	<pre>PolynomialFeatures(degree=2).fit_transform(X)</pre>
<code>PowerTransformer()</code>	Gaussianize skewed features	<pre>PowerTransformer().fit_transform(X)</pre>
<code>ColumnTransformer()</code>	Apply transforms per column	<pre>ColumnTransformer([('sc',StandardScaler(),[0,1])])</pre>

Models — Regression, Classification & Clustering

Function / Syntax	Description	Code Example
<code>LinearRegression()</code>	Ordinary least squares	<pre>LinearRegression().fit(X_tr, y_tr)</pre>
<code>Ridge(alpha)</code>	L2 regularized regression	<pre>Ridge(alpha=1.0).fit(X_tr, y_tr)</pre>
<code>Lasso(alpha)</code>	L1 regularized regression	<pre>Lasso(alpha=0.1).fit(X_tr, y_tr)</pre>
<code>ElasticNet()</code>	L1+L2 regularized regression	<pre>ElasticNet(l1_ratio=0.5).fit(X_tr, y_tr)</pre>
<code>LogisticRegression()</code>	Binary/multi-class classifier	<pre>LogisticRegression(C=1.0, max_iter=1000)</pre>
<code>SVC(kernel)</code>	Support vector classifier	<pre>SVC(kernel='rbf', C=1.0, gamma='scale')</pre>
<code>SVR()</code>	Support vector regressor	<pre>SVR(kernel='rbf', C=100)</pre>
<code>KNeighborsClassifier(k)</code>	K-nearest neighbors	<pre>KNeighborsClassifier(n_neighbors=5)</pre>
<code>DecisionTreeClassifier()</code>	Decision tree classifier	<pre>DecisionTreeClassifier(max_depth=5)</pre>

Function / Syntax	Description	Code Example
<code>RandomForestClassifier()</code>	Ensemble of decision trees	<code>RandomForestClassifier(n_estimators=100)</code>
<code>GradientBoostingClassifier()</code>	Gradient boosting trees	<code>GradientBoostingClassifier(n_estimators=200)</code>
<code>XGBClassifier()</code>	Extreme gradient boosting	<code>XGBClassifier(n_estimators=200, learning_rate=0.1)</code>
<code>AdaBoostClassifier()</code>	Adaptive boosting	<code>AdaBoostClassifier(n_estimators=50)</code>
<code>BaggingClassifier()</code>	Bootstrap aggregating	<code>BaggingClassifier(n_estimators=10)</code>
<code>GaussianNB()</code>	Naïve Bayes classifier	<code>GaussianNB().fit(X_tr, y_tr)</code>
<code>MLPClassifier()</code>	Neural network classifier	<code>MLPClassifier(hidden_layer_sizes=(100,50))</code>
<code>KMeans(k)</code>	K-Means clustering	<code>KMeans(n_clusters=5, random_state=42)</code>
<code>DBSCAN()</code>	Density-based clustering	<code>DBSCAN(eps=0.5, min_samples=5)</code>
<code>AgglomerativeClustering()</code>	Hierarchical clustering	<code>AgglomerativeClustering(n_clusters=3)</code>
<code>IsolationForest()</code>	Anomaly detection	<code>IsolationForest(contamination=0.05)</code>

Dimensionality Reduction & Feature Selection

Function / Syntax	Description	Code Example
<code>PCA(n)</code>	Principal Component Analysis	<code>pca = PCA(n_components=2); X_red = pca.fit_transform(X)</code>
<code>TruncatedSVD()</code>	SVD for sparse matrices	<code>TruncatedSVD(n_components=50).fit_transform(X)</code>
<code>TSNE(n)</code>	t-SNE for visualization	<code>TSNE(n_components=2, perplexity=30).fit_transform(X)</code>
<code>UMAP(n)</code>	UMAP dimensionality reduction	<code>umap.UMAP(n_components=2).fit_transform(X)</code>
<code>SelectKBest()</code>	Select k best features	<code>SelectKBest(f_classif, k=10).fit_transform(X, y)</code>
<code>RFE()</code>	Recursive feature elimination	<code>RFE(estimator=clf, n_features_to_select=5)</code>
<code>VarianceThreshold()</code>	Remove low-variance features	<code>VarianceThreshold(threshold=0.01)</code>

Evaluation, Tuning & Pipelines

Function / Syntax	Description	Code Example
<code>accuracy_score()</code>	Classification accuracy	<code>accuracy_score(y_te, y_pred)</code>
<code>precision_score()</code>	Precision	<code>precision_score(y_te, y_pred, average='macro')</code>
<code>recall_score()</code>	Recall / sensitivity	<code>recall_score(y_te, y_pred)</code>
<code>f1_score()</code>	F1 harmonic mean of P & R	<code>f1_score(y_te, y_pred, average='weighted')</code>
<code>roc_auc_score()</code>	AUC of ROC curve	<code>roc_auc_score(y_te, y_proba)</code>
<code>confusion_matrix()</code>	Confusion matrix	<code>confusion_matrix(y_te, y_pred)</code>
<code>classification_report()</code>	Full classification metrics	<code>print(classification_report(y_te, y_pred))</code>
<code>mean_squared_error()</code>	MSE for regression	<code>mean_squared_error(y_te, y_pred, squared=False) # RMSE</code>

Function / Syntax	Description	Code Example
<code>mean_absolute_error()</code>	MAE for regression	<code>mean_absolute_error(y_te, y_pred)</code>
<code>r2_score()</code>	R-squared coefficient	<code>r2_score(y_te, y_pred)</code>
<code>cross_val_score()</code>	K-fold cross-validation	<code>cross_val_score(clf, X, y, cv=5, scoring='f1')</code>
<code>GridSearchCV()</code>	Exhaustive hyperparameter search	<code>GridSearchCV(clf, param_grid, cv=5, scoring='roc_auc')</code>
<code>RandomizedSearchCV()</code>	Random hyperparameter search	<code>RandomizedSearchCV(clf, params, n_iter=50, cv=5)</code>
<code>Pipeline(steps)</code>	Chain transforms + estimator	<code>Pipeline([('sc', StandardScaler()), ('clf', SVC())])</code>



Tensor Creation

Function / Syntax	Description	Code Example
<code>torch.tensor(data)</code>	Create tensor from data	<code>x = torch.tensor([[1.,2.],[3.,4.]])</code>
<code>torch.zeros(shape)</code>	Zeros tensor	<code>torch.zeros(3, 4, dtype=torch.float32)</code>
<code>torch.ones(shape)</code>	Ones tensor	<code>torch.ones(3, 4)</code>
<code>torch.rand(shape)</code>	Uniform random [0,1)	<code>torch.rand(100, 10)</code>
<code>torch.randn(shape)</code>	Standard normal random	<code>torch.randn(batch_size, 128)</code>
<code>torch.arange()</code>	Range tensor	<code>torch.arange(0, 10, step=2)</code>
<code>torch.linspace()</code>	Evenly spaced tensor	<code>torch.linspace(0, 1, 100)</code>
<code>torch.eye(n)</code>	Identity matrix	<code>torch.eye(4)</code>
<code>torch.from_numpy(arr)</code>	Tensor from NumPy array	<code>x = torch.from_numpy(np_array)</code>
<code>tensor.numpy()</code>	Convert tensor to NumPy	<code>arr = x.cpu().numpy()</code>
<code>tensor.item()</code>	Python scalar from 1-elem tensor	<code>loss_val = loss.item()</code>
<code>tensor.tolist()</code>	Convert tensor to Python list	<code>x.tolist()</code>

Tensor Operations & Manipulation

Function / Syntax	Description	Code Example
<code>tensor.shape / .size()</code>	Tensor dimensions	<code>x.shape # torch.Size([32, 128])</code>
<code>tensor.dtype / .device</code>	Data type / device	<code>x.dtype; x.device # cuda:0</code>
<code>tensor.to(device)</code>	Move to device	<code>x = x.to('cuda')</code>
<code>tensor.cuda() / cpu()</code>	Move to GPU / CPU	<code>x.cuda(); x.cpu()</code>
<code>tensor.view(shape)</code>	Reshape (must be contiguous)	<code>x.view(batch_size, -1)</code>
<code>tensor.reshape(shape)</code>	Reshape (safe)	<code>x.reshape(-1, 28*28)</code>
<code>tensor.permute(dims)</code>	Reorder dimensions	<code>x.permute(0, 2, 3, 1) # NCHW -> NHWC</code>
<code>tensor.squeeze(dim)</code>	Remove size-1 dimension	<code>x.squeeze(0)</code>
<code>tensor.unsqueeze(dim)</code>	Add size-1 dimension	<code>x.unsqueeze(1)</code>

Function / Syntax	Description	Code Example
<code>torch.cat(tensors, dim)</code>	Concatenate along axis	<code>torch.cat([a, b], dim=0)</code>
<code>torch.stack(tensors)</code>	Stack along new axis	<code>torch.stack([a, b], dim=1)</code>
<code>torch.matmul(a, b)</code>	Matrix multiplication	<code>out = torch.matmul(W, x) + b</code>
<code>tensor.T</code> / <code>.t()</code>	Transpose 2D tensor	<code>W.T</code>
<code>torch.clamp()</code>	Clip tensor values	<code>torch.clamp(x, min=0, max=1)</code>
<code>tensor.detach()</code>	Detach from computation graph	<code>y = x.detach()</code>
<code>tensor.requires_grad_(True)</code>	Enable gradient tracking	<code>x.requires_grad_(True)</code>
<code>torch.no_grad()</code>	Context: disable gradients	<code>with torch.no_grad(): out = model(x)</code>

Neural Network Layers & Losses

Function / Syntax	Description	Code Example
<code>nn.Module</code>	Base class for all models	<code>class Net(nn.Module): def __init__(self): super().__init__()</code>
<code>nn.Linear(in, out)</code>	Fully-connected layer	<code>nn.Linear(784, 256)</code>
<code>nn.Conv2d(in,out,k)</code>	2D convolution	<code>nn.Conv2d(3, 64, kernel_size=3, padding=1)</code>
<code>nn.MaxPool2d(k)</code>	Max pooling	<code>nn.MaxPool2d(kernel_size=2, stride=2)</code>
<code>nn.BatchNorm2d(c)</code>	Batch normalization (2D)	<code>nn.BatchNorm2d(64)</code>
<code>nn.Dropout(p)</code>	Dropout regularization	<code>nn.Dropout(p=0.5)</code>
<code>nn.Embedding(n,d)</code>	Embedding lookup table	<code>nn.Embedding(vocab_size, embed_dim)</code>
<code>nn.LSTM(in,h)</code>	Long Short-Term Memory	<code>nn.LSTM(input_size=128, hidden_size=256, num_layers=2)</code>
<code>nn.GRU(in,h)</code>	Gated Recurrent Unit	<code>nn.GRU(128, 256, batch_first=True)</code>
<code>nn.MultiheadAttention()</code>	Multi-head self-attention	<code>nn.MultiheadAttention(embed_dim=512, num_heads=8)</code>
<code>nn.TransformerEncoder()</code>	Transformer encoder stack	<code>nn.TransformerEncoder(encoder_layer, num_layers=6)</code>
<code>nn.Sequential(*layers)</code>	Chain layers sequentially	<code>nn.Sequential(nn.Linear(784,256), nn.ReLU(), nn.Linear(256,10))</code>
<code>nn.ReLU()</code>	ReLU activation	<code>nn.ReLU()</code>
<code>nn.Sigmoid()</code>	Sigmoid activation	<code>nn.Sigmoid()</code>
<code>nn.Softmax(dim)</code>	Softmax activation	<code>nn.Softmax(dim=1)</code>
<code>nn.CrossEntropyLoss()</code>	Classification loss	<code>criterion = nn.CrossEntropyLoss()</code>
<code>nn.BCEWithLogitsLoss()</code>	Binary classification loss	<code>nn.BCEWithLogitsLoss()</code>
<code>nn.MSELoss()</code>	Mean squared error loss	<code>criterion = nn.MSELoss()</code>
<code>nn.L1Loss()</code>	Mean absolute error loss	<code>nn.L1Loss()</code>

Optimizers & Training Loop

Function / Syntax	Description	Code Example
<code>optim.Adam()</code>	Adam optimizer	<code>optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)</code>
<code>optim.SGD()</code>	Stochastic gradient descent	<code>optim.SGD(model.parameters(), lr=0.01, momentum=0.9)</code>

Function / Syntax	Description	Code Example
<code>optim.AdamW()</code>	Adam with weight decay fix	<code>optim.AdamW(model.parameters(), lr=2e-4)</code>
<code>optim.RMSprop()</code>	RMSprop optimizer	<code>optim.RMSprop(model.parameters(), lr=1e-3)</code>
<code>optimizer.zero_grad()</code>	Clear accumulated gradients	<code>optimizer.zero_grad()</code>
<code>loss.backward()</code>	Compute gradients	<code>loss.backward()</code>
<code>optimizer.step()</code>	Update model parameters	<code>optimizer.step()</code>
<code>nn.utils.clip_grad_norm_()</code>	Clip gradient norms	<code>nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)</code>
<code>lr_scheduler.StepLR()</code>	Decay LR every n steps	<code>StepLR(optimizer, step_size=10, gamma=0.5)</code>
<code>lr_scheduler.CosineAnnealingLR()</code>	Cosine LR annealing	<code>CosineAnnealingLR(optimizer, T_max=100)</code>
<code>model.train()</code>	Set to training mode	<code>model.train()</code> # enables dropout, batchnorm
<code>model.eval()</code>	Set to eval mode	<code>model.eval()</code> # disables dropout, batchnorm
<code>torch.save(state_dict)</code>	Save model weights	<code>torch.save(model.state_dict(), 'model.pt')</code>
<code>torch.load()</code>	Load model weights	<code>model.load_state_dict(torch.load('model.pt'))</code>

Data Loading & Transforms

Function / Syntax	Description	Code Example
<code>Dataset.__getitem__()</code>	Custom dataset indexing	<code>def __getitem__(self, idx): return X[idx], y[idx]</code>
<code>Dataset.__len__()</code>	Dataset length	<code>def __len__(self): return len(self.data)</code>
<code>DataLoader(ds, batch)</code>	Batch + shuffle dataset	<code>DataLoader(ds, batch_size=32, shuffle=True, num_workers=4)</code>
<code>transforms.Compose()</code>	Chain image transforms	<code>transforms.Compose([transforms.ToTensor(), transforms.Normalize()])</code>
<code>transforms.ToTensor()</code>	PIL image to tensor	<code>transforms.ToTensor()</code>
<code>transforms.Normalize()</code>	Normalize image tensor	<code>transforms.Normalize(mean=[0.5], std=[0.5])</code>
<code>torchvision.datasets.*</code>	Built-in datasets	<code>datasets.MNIST('./data', train=True, download=True)</code>
<code>torchvision.models.*</code>	Pretrained models	<code>models.resnet50(pretrained=True)</code>



LangGraph

Core State & Graph Primitives

Function / Syntax	Description	Code Example
<code>StateGraph (schema)</code>	Create graph with typed state	<pre>from langgraph.graph import StateGraph graph = StateGraph(MyState)</pre>
<code>MessageGraph ()</code>	Graph using message list state	<pre>from langgraph.graph import MessageGraph graph = MessageGraph()</pre>
<code>TypedDict</code>	Define state schema	<pre>class MyState(TypedDict): messages: list count: int</pre>
<code>Annotated[T, reducer]</code>	Field with custom reducer	<pre>messages: Annotated[list, operator.add]</pre>
<code>operator.add</code>	Append-to-list reducer	<pre>messages: Annotated[list, operator.add]</pre>
<code>END</code>	Terminal node constant	<pre>graph.add_edge('node', END)</pre>
<code>START</code>	Entry node constant	<pre>graph.add_edge(START, 'first_node')</pre>

Graph Building & Execution

Function / Syntax	Description	Code Example
<code>graph.add_node(name, fn)</code>	Register a node function	<pre>graph.add_node('agent', call_llm)</pre>
<code>graph.add_edge(a, b)</code>	Static directional edge	<pre>graph.add_edge('tool', 'agent')</pre>
<code>graph.add_conditional_edges()</code>	Route based on output	<pre>graph.add_conditional_edges('agent', route_fn, {'tool':'tool', 'end':END})</pre>
<code>graph.set_entry_point()</code>	Set starting node (legacy)	<pre>graph.set_entry_point('agent')</pre>
<code>graph.set_finish_point()</code>	Set ending node (legacy)	<pre>graph.set_finish_point('output')</pre>
<code>graph.compile()</code>	Compile to runnable	<pre>app = graph.compile()</pre>
<code>graph.compile(checkpointer)</code>	Compile with memory	<pre>app = graph.compile(checkpointer=MemorySaver())</pre>
<code>app.invoke(input)</code>	Run graph synchronously	<pre>result = app.invoke({'messages': [HumanMessage('Hi')]}))</pre>
<code>app.stream(input)</code>	Stream graph step by step	<pre>for chunk in app.stream(input, config): print(chunk)</pre>
<code>app.ainvoke(input)</code>	Async graph invocation	<pre>result = await app.ainvoke(input, config)</pre>
<code>app.astream(input)</code>	Async streaming	<pre>async for chunk in app.astream(input, config): ...</pre>

Memory & Persistence

Function / Syntax	Description	Code Example
<code>MemorySaver ()</code>	In-memory thread checkpointer	<pre>checkpointer = MemorySaver()</pre>
<code>SqliteSaver.from_conn()</code>	SQLite persistent memory	<pre>SqliteSaver.from_conn_string('memory.db')</pre>
<code>PostgresSaver()</code>	Postgres persistent memory	<pre>PostgresSaver.from_conn_string(conn_str)</pre>
<code>thread_id in config</code>	Isolate conversation sessions	<pre>config = {'configurable': {'thread_id': 'user-123'}}</pre>
<code>app.get_state(config)</code>	Read current graph state	<pre>state = app.get_state(config)</pre>
<code>app.update_state(cfg, val)</code>	Manually patch state	<pre>app.update_state(config, {'count': 5})</pre>
<code>app.get_state_history(cfg)</code>	Full state history	<pre>for s in app.get_state_history(config): print(s)</pre>

Human-in-the-Loop (HITL)

Function / Syntax	Description	Code Example
<code>interrupt_before=['node']</code>	Pause before node runs	<code>app = graph.compile(interrupt_before='human_review')</code>
<code>interrupt_after=['node']</code>	Pause after node runs	<code>app = graph.compile(interrupt_after='agent')</code>
<code>app.update_state(cfg, v)</code>	Inject human feedback	<code>app.update_state(config, {'approved': True})</code>
<code>NodeInterrupt(msg)</code>	Raise interrupt from node	<code>raise NodeInterrupt('Need human approval')</code>
<code>app.invoke(None, config)</code>	Resume after interrupt	<code>app.invoke(None, config) # resumes where paused</code>

Tools & Messages

Function / Syntax	Description	Code Example
<code>ToolNode(tools)</code>	Built-in node to execute tools	<code>tool_node = ToolNode([search, calculator])</code>
<code>tools_condition(state)</code>	Route to tool or END	<code>graph.add_conditional_edges('agent', tools_condition)</code>
<code>@tool decorator</code>	Define a LangChain tool	<code>@tool def search(q: str) -> str: return web_search(q)</code>
<code>bind_tools(tools)</code>	Bind tools to LLM	<code>llm_with_tools = llm.bind_tools([search, calculator])</code>
<code>ToolMessage</code>	Result message from a tool	<code>ToolMessage(content=result, tool_call_id=call.id)</code>
<code>HumanMessage(content)</code>	User message	<code>HumanMessage(content='What is the weather?')</code>
<code>AIMessage(content)</code>	LLM response message	<code>AIMessage(content='The weather is...')</code>
<code>SystemMessage(content)</code>	System prompt message	<code>SystemMessage(content='You are a helpful assistant')</code>

Subgraphs, Fan-out & Advanced Routing

Function / Syntax	Description	Code Example
<code>graph.compile().as_runnable()</code>	Use subgraph as node	<code>parent.add_node('sub', subgraph.compile())</code>
<code>Send(node, state)</code>	Fan-out: send to multiple	<code>return [Send('process', {'item': x}) for x in items]</code>
<code>Command(goto, update)</code>	Dynamic routing + update	<code>return Command(goto='agent', update={'count': n+1})</code>
<code>InjectedState</code>	Pass full state to tool	<code>def tool(query: str, state: Annotated[State, InjectedState])</code>
<code>RunnableConfig</code>	Pass config to nodes	<code>def node(state, config: RunnableConfig): ...</code>